1. Project Title

Adaptive TCP Congestion Control Decision Engine Using Mininet and Machine Learning

---

A Data-Driven System for Selecting the Best-Performing TCP Congestion Control Algorithm under Varying Network Conditions

Team Members

Rohan Kumar – 24BDS0463

Shreyans Bohra – 24BCT0013

Course: Computer Networks (BCSE308P)

Review 1 – Problem Identification & Planning

2. Introduction

TCP stands for Transmission Control Protocol, which is a protocol that ensures the reliable delivery of data over IP networks. One of the important functions of TCP is congestion control, which regulates the rate at which a sender transmits data to reduce the likelihood of network congestion.

TCP uses different congestion control algorithms, including Reno, Cubic, and BBR which operate under varying conditions, which include varying amounts of bandwidth, delay and packet loss. This project investigates how changes in network conditions affect the relative performance of these algorithms.

A Mininet network will be configured with controlled variations in bandwidth, delay, and packet loss. TCP Reno, Cubic, and BBR will then be benchmarked under these conditions. The results from the benchmarks will then be used as the dataset for the machine-learning-based decision engine.

The project starts off with networking and ends up using machine learning as the decision engine.

3. Problem Statement

TCP congestion control algorithms do not have the same performance in different types of network environments. A congestion control algorithm suitable for one environment may not work well in another network with different bandwidth, latency, and packet loss.

In a traditional approach, a TCP congestion control algorithm is chosen via the operating system's settings, as opposed to selecting it based on the results of measuring network conditions of each flow by the decision engine. Therefore, the conventional configuration does not use measured network conditions as an explicit input to a learning-based decision process for selecting among TCP variants.

The research question to be investigated in this project is: Is it possible to predict the best throughput performance of one of the TCP Reno, Cubic, and BBR based on experimentally measured network conditions?

This question will be investigated by benchmarking all three variants in a controlled environment, identifying the best-performing variant from the measured results, and training a classifier to predict that choice for unseen conditions.

4. Motivation

- Various permutations of bandwidth, delay, and packet loss will affect the performance of Reno, Cubic, and BBR differently.
- Mininet and Linux tc/netem allow for an environment where such conditions can be simulated consistently.
- The experiment will generate a project-specific data set instead of only using a publicly available one.
- A classification model can map measurable network conditions to a predicted TCP variant.

This research does not aim at developing a new TCP algorithm. The goal is to explore whether an intelligent choice between existing TCP variants is better than using just one variant.

5. Objectives

1. Build a network topology H1-S1-H2 in Mininet.
2. Use Linux tc, with netem for delay and packet-loss emulation and appropriate traffic-control disciplines for bandwidth limitation. Collect the baseline RTT before TCP benchmarking for each configured network condition.
3. Benchmark TCP Reno, Cubic, and BBR with iperf3 under all configured network conditions.
4. Collect throughput and retransmissions and keep the data in a raw format.
5. Gather the data per network condition and designate the TCP protocol with maximum throughput as the best performing one.
6. Train a Random Forest classifier with bandwidth, delays, packet loss, and baseline RTT as features.
7. Test the model with network conditions not used during training.
8. Compare ML-based approach with static Reno, Cubic, and BBR TCP.

6. Existing System

The algorithm for TCP congestion control is chosen by the configuration of the operating system or application, and not based on a learning engine that determines an algorithm according to the measurements of the network.

- **TCP Reno** – a conventional congestion control algorithm that operates on loss.
- **TCP Cubic** – a congestion control algorithm that is effective on high-bandwidth networks.
- **TCP BBR** – a model-based congestion control algorithm that estimates bottleneck bandwidth and RTT and uses these estimates to control transmission.

There is literature that studies learning-based congestion control and dynamic congestion control algorithm selection as well. While Remy learns the congestion control rules and Antelope examines dynamic selection of congestion control algorithms, these studies show that a data-based approach is possible; however, their focus is different from our decision-engine experiment study.

7. Limitations of Existing System

- A single configuration of a TCP variant might not always yield the best results in all possible combinations of bandwidth, delay, and loss rate.
- There is no explicit learning involved regarding the relationship between network parameters and the performance of various TCP variants.
- The manual choice method demands that the user have some knowledge about what TCP variant should ideally be used in a certain scenario.
- Trial under limited number of conditions does not form the basis for any kind of selection process.
- Converting a TCP into another TCP variant and switching dynamically at the kernel level add implementation complications beyond the scope of this two-student, six-week course project.

8. Proposed System

The proposed decision engine is experimental. It includes a Python automation script which drives the experiments in a Mininet topology and applies network impairments using tc/netem. First, the baseline RTT is measured for each network condition. The condition is then separately evaluated with Reno, Cubic and BBR using iperf3.

In each condition-variant combination throughput and retransmission measurements are taken. Three runs are intended to average out the noise; the median throughput will be used for choosing the best performing variant. The best performing variant is selected based on the maximum median throughput in Reno, Cubic and BBR. The label for the network condition will be the best performing variant.

Two logical datasets will be kept:

- **Dataset A - Raw Benchmark Dataset**: one record for each condition and TCP variant; records include bandwidth, delay, loss, TCP variant, throughput, RTT, and retransmissions.
- **Dataset B - ML Dataset**: one record for each network condition including bandwidth, delay, loss, baseline RTT, measured throughput for Reno/Cubic/BBR and best performing variant.

Random Forest classification model is going to be trained using bandwidth, delay, packet loss and baseline RTT as features, and Reno, Cubic or BBR as multiclass output. The measured throughput values of the three variants will be retained for label generation and performance analysis but will not be used as classifier inputs, since they are only available after running the corresponding TCP experiments. The evaluation process will include held-out network conditions and not just random train-test split.

The final comparison will compare four approaches: always Reno, always Cubic, always BBR and ML Decision Engine. This way the assessment becomes a Computer Networks performance benchmarking exercise, not just ML accuracy evaluation.

9. Novelty / Innovation

The project contribution is the combination of controlled TCP experimentation with a classification-based decision engine for selecting among existing congestion control algorithms.

- Self-created network dataset from controlled Mininet experiments.
- Conditional selection of techniques from measurable network parameters.
- Evaluation on held-out conditions to assess performance on unseen network conditions.
- Comparison of ML technique with TCP baselines in controlled settings.
- Controlled experimental setting suitable for two-member six-week coursework project.

The current project is a TCP selection framework based on experimental validation but it does not propose any novel congestion-control algorithms.

10. Literature Survey

10.1 TCP Ex Machina: Computer-Generated Congestion Control (Remy)

This paper demonstrated that computer programs could be used to generate congestion control policies based on network models and a cost function. However, the learned policy itself is a congestion control policy and not a policy for selecting from amongst the different TCP policies.

[1] K. Winstein and H. Balakrishnan, "TCP Ex Machina: Computer-Generated Congestion Control," Proceedings of the ACM SIGCOMM 2013 Conference, pp. 123–134, 2013.

10.2 A Machine Learning Approach to Improve Congestion Control over Wireless Computer Networks

Geurts, El Khayat, and Leduc considered using machine learning for classifying congestion-caused packet losses and wireless link errors. This work involves supervised learning applied to the problem of TCP congestion control, but concentrates on packet-loss classification and not congestion control per se.

[2] P. Geurts, I. El Khayat, and G. Leduc, "A Machine Learning Approach to Improve Congestion Control over Wireless Computer Networks," Proceedings of the Fourth IEEE International Conference on Data Mining (ICDM'04), pp. 383–386, 2004.

10.3 Antelope: A Framework for Dynamic Selection of Congestion Control Algorithms

Antelope framework was developed by Zhou et al. where a trained machine learning model helps in dynamic selection of congestion control algorithm. This work is closely related to the proposed project. The main difference is that the proposed project first develops and evaluates the decision engine in a controlled Mininet environment rather than implementing live kernel-level dynamic switching.

[3] J. Zhou, X. Qiu, Z. Li, G. Tyson, Q. Li, J. Duan, and Y. Wang, "Antelope: A Framework for Dynamic Selection of Congestion Control Algorithms," Proceedings of the 29th IEEE International Conference on Network Protocols (ICNP), pp. 1–11, 2021. doi: 10.1109/ICNP52444.2021.9651912.

10.4 When Machine Learning Meets Congestion Control: A Survey and Comparison

The authors conducted an analysis of machine learning methods used for congestion control and elaborated on supervised, unsupervised, and reinforcement learning-based approaches. In this context, the current work is viewed as a supervised classification method, which learns from measurements obtained from the network.

[4] H. Jiang, Q. Li, Y. Jiang, G. Shen, R. Sinnott, C. Tian, and M. Xu, "When Machine Learning Meets Congestion Control: A Survey and Comparison," Computer Networks, vol. 192, article 108033, 2021. doi: 10.1016

10.5 Reproducible Measurements of TCP BBR Congestion Control

TCP BBR was investigated by Jaeger et al. via reproducible experimental measurements and was benchmarked against conventional TCP congestion control algorithms. This study promotes the idea of evaluating TCP efficiency through controlled measurements and is highly relevant to our plan to use Mininet and iperf3 methodology.

[5] B. Jaeger, D. Scholz, D. Raumer, F. Geyer, and G. Carle, "Reproducible Measurements of TCP BBR Congestion Control," Computer Communications, vol. 144, pp. 31-43, 2019. doi: 10.1

10.6 Intelligent TCP Congestion Control Policy Optimization

Shi and Wang examined the use of reinforcement learning and Mininet to optimize intelligent TCP congestion control policies. This research shows that the use of Mininet for evaluating learning-based TCP congestion control is feasible. In contrast, the current project employs a supervised classification approach to choose between different TCP implementations.

[6] H. Shi and J. Wang, "Intelligent TCP Congestion Control Policy Optimization," Applied Sciences, vol. 13, no. 11, article 6644, 2023. doi: 10.3390/app13116644.

10.7 Identified Research Gap

In the selected paper, it can be observed that machine learning has been utilized for the purposes of generating congestion control policies, for classification of behaviors related to congestion, and algorithm selection. The proposed project focuses on a narrower and experimentally feasible problem: developing a lightweight decision engine that selects among Reno, Cubic, and BBR using a self-generated Mininet dataset.

11. Tools and Technologies

Category	Tool / Purpose
Operating System / Environment	Ubuntu Linux under WSL2 on Windows, or a native/virtual Ubuntu environment if required for BBR support.
Network Emulation	Mininet with a simple H1-S1-H2 topology.
Switching	Open vSwitch which is used in the Mininet environment.
Traffic Shaping	Linux tc with netem for delay and packet-loss emulation, along with appropriate traffic-control disciplines for bandwidth limitation.
TCP Variants	TCP Reno, TCP Cubic, and TCP BBR.
Traffic Creation & Measurement	iperf3 to measure TCP throughput and retransmissions; ping to get baseline RTT, ss to get TCP socket statistics and some other connection statistics.
Automation	Python script for conducting experiments and creating CSV files from results.
Data processing	Pandas to clean, aggregate and prepare data.
Machine Learning	Scikit-learn, primarily Random Forest, with Decision Tree as a baseline.
Visualization	Matplotlib to draw performance charts, confusion matrix and features importance.
Source code version control	Git and GitHub for source-code and experiment-script management.

Verification of BBR availability will happen early since the entire experiment depends on all three types of TCP. In case the WSL2 Kernel is unable to offer BBR, an Ubuntu virtual machine can be used as a replacement.

12. Initial System Architecture / Workflow

The design takes a pipeline approach, starting from network emulation all the way to TCP recommendation.

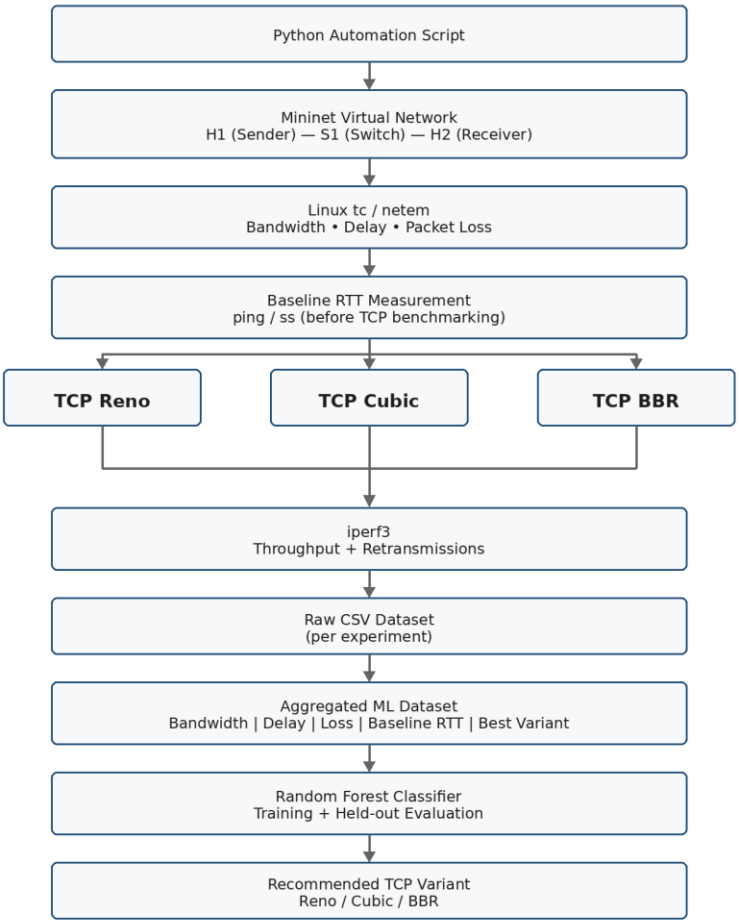


***Figure 1**. Design of Adaptive TCP Congestion Control Decision Engine and its workflow.*

Workflow: Python script orchestrates the test; Mininet sets up H1-S1-H2 network topology; tc/netem applies bandwidth, delay, and packet-loss configuration; baseline round-trip time (RTT) is recorded before TCP benchmarking; Reno, Cubic, and BBR are benchmarked using iperf3; data is collected, consolidated, and stored; training is done for Random Forest with held-out evaluation; and the model selects the TCP variant.

Preliminary test grid: 6 bandwidths (1, 5, 10, 20, 50, 100 Mbps) × 5 delays (5, 20, 50, 100, 200 ms) × 5 losses (0, 1, 2, 3, 5%) = 150 network conditions, each benchmarked using 3 TCP variants = 450 TCP tests per experiment run. Three repeated experiment runs yield a total of 1,350 TCP runs required for the full experiment. Smaller 54-TCP-run pilot will be performed first to validate the automation pipeline.

13. Project Timeline

Week	Planned Activities	Target
1	Learn about TCP congestion control and Mininet/iperf3. Setup Ubuntu/WSL2, verify Reno/Cubic/BBR, and create H1-S1-H2 topology.	A successful iperf3 test, confirming BBR.
2	Create tc/netem configuration for bandwidth, latency, and loss. Automate the baseline RTT and conduct pilot experiment of 54 tests.	Pilot runs through end-to-end.
3	Conduct experiments of all 150 conditions for 3 variants and 3 iterations each. Create raw and aggregate datasets	Completion of experimental dataset.
4	Data analysis and exploration. Creation of initial graphs. Training of baseline Decision Tree and Random Forest models.	First functional ML model.
5	Model optimization, out-of-sample evaluation, and comparison of ML selection with baseline algorithms.	Decide on whether ML selection has any advantage.
6	Development of prediction interface, finalization of visuals and documentation.	Completion of final product, report, and presentation.

14. Responsibilities of Team Members

Rohan Kumar – 24BDS0463	Shreyans Bohra – 24BCT0013
Networking and Experiments • Mininet configuration and topology of H1-S1-H2 • tc/netem configuration • TCP Reno/Cubic/BBR configuration • Configuration of iperf3 and baseline RTT • Conducting experiments multiple times and creation of raw CSV files • Verifying networking experiments	**ML and Analysis** • Cleaning and aggregating data set • Preparation of features and labels • Baseline of decision tree and random forest • Held out evaluation and comparison of fixed algorithms • Graph plotting and analysis of models • Predictive Interface

Both will understand the full project cycle and work together on literature review, integration, testing, documentation and presentation. Responsibilities are divided for efficiency and balanced contribution.